



WOLLO University

Department of software engineering

Exit multiple choice questions for

Data structure and algorithm

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

1 In a heap with n elements with the smallest element at the root, the 7th smallest element can be found in time (GATE CS 2003)

- a) $\Theta(n \log n)$
- b) $\Theta(n)$
- c) $\Theta(\log n)$
- d) $\Theta(1)$

Answer(d)

The 7th smallest element must be in first 7 levels. Total number of nodes in any Binary Heap in first 7 levels is at most $1 + 2 + 4 + 8 + 16 + 32 + 64$ which is a constant. Therefore we can always find 7th smallest element in $\Theta(1)$ time.

2. Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers. What is the in-order traversal sequence of the resultant tree? (GATE CS 2003)

- a) 7 5 1 0 3 2 4 6 8 9
- b) 0 2 4 3 1 6 5 9 8 7
- c) 0 1 2 3 4 5 6 7 8 9
- d) 9 8 6 4 2 3 0 1 5 7

Answer (c)

In-order traversal of a BST gives elements in increasing order. So answer c is correct without any doubt.

3. Let S be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that Push and Pop operation take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of Push(m) to the start of the pop operation that removes m from S . The average stack-life of an element of this stack is (GATE CS 2003)

- a) $n(X + Y)$
- b) $3Y + 2X$
- c) $n(X + Y) - X$
- d) $Y + 2X$

Answer(c)

Following questions have been asked in GATE CS exam.

1. Consider the following functions

$$f(n) = 3n^{\sqrt{n}}$$

$$g(n) = 2^{\sqrt{n} \log_2 n}$$

$$h(n) = n!$$

Which of the following is true? (GATE CS 2000)

- (a) $h(n)$ is $O(f(n))$
- (b) $h(n)$ is $O(g(n))$
- (c) $g(n)$ is not $O(f(n))$
- (d) $f(n)$ is $O(g(n))$

Answer (d)

$$g(n) = 2^{\sqrt{n} \log_2 n} = n^{\sqrt{n}}$$

$f(n)$ and $g(n)$ are of same asymptotic order and following statements are true.

$$f(n) = O(g(n))$$

$$g(n) = O(f(n)).$$

(a) and (b) are false because $n!$ is of asymptotically higher order than $n^{\sqrt{n}}$.

2. Let G be an undirected connected graph with distinct edge weight. Let $e_{\max}$ be the edge with maximum weight and $e_{\min}$ the edge with minimum weight. Which of the following statements is false? (GATE CS 2000)

- (a) Every minimum spanning tree of G must contain $e_{\min}$
- (b) If $e_{\max}$ is in a minimum spanning tree, then its removal must disconnect G
- (c) No minimum spanning tree contains $e_{\max}$
- (d) G has a unique minimum spanning tree

Answer (c)

(a) and (b) are always true.

(c) is false because (b) is true.

(d) is true because all edge weights are distinct for G .

3. Let G be an undirected graph. Consider a depth-first traversal of G , and let T be the resulting depth-first search tree. Let u be a vertex in G

and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE CS 2000)

- (a) $\{u, v\}$ must be an edge in G , and u is a descendant of v in T
- (b) $\{u, v\}$ must be an edge in G , and v is a descendant of u in T
- (c) If $\{u, v\}$ is not an edge in G then u is a leaf in T
- (d) If $\{u, v\}$ is not an edge in G then u and v must have the same parent in T

Answer (c)

See <http://geeksquiz.com/data-structures-graph-question-20/> for explanation.

4. Consider an undirected unweighted graph G . Let a breadth-first traversal of G be done starting from a node r . Let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v respectively, in G . If u is visited before v during the breadth-first traversal, which of the following statements is correct? (GATE CS 2001)

- a) $d(r, u) < d(r, v)$
- b) $d(r, u) > d(r, v)$
- c) $d(r, u) \leq d(r, v)$
- d) None of the above

Answer (c) $d(r, u)$ and $d(r, v)$ will be equal when u and v are at same level, otherwise $d(r, u)$ will be less than $d(r, v)$

5. How many undirected graphs (not necessarily connected) can be constructed out of a given set $V = \{V_1, V_2, \dots, V_n\}$ of n vertices? (GATE CS 2001)

- a) $n(n-1)/2$
- b) 2^n
- c) $n!$
- d) $2^{n(n-1)/2}$

Answer (d)

In an undirected graph, there can be maximum $n(n-1)/2$ edges. We can choose to have (or not have) any of the $n(n-1)/2$ edges. So, total number of undirected graphs with n vertices is $2^{n(n-1)/2}$.

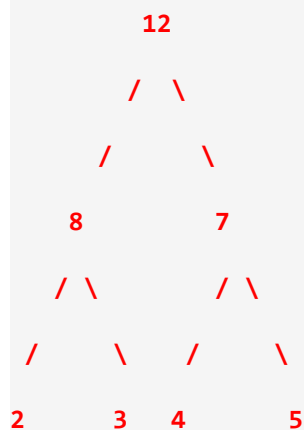
Following questions have been asked in GATE CS 2006 exam.

1. In a binary max heap containing n numbers, the smallest element can be found in time (GATE CS 2006)

- (A) $O(n)$
- (B) $O(\log n)$
- (C) $O(\log \log n)$
- (D) $O(1)$

Answer (A)

In a max heap, the smallest element is always present at a leaf node. So we need to check for all leaf nodes for the minimum value. Worst case complexity will be $O(n)$



2. A scheme for storing binary trees in an array X is as follows. Indexing of X starts at 1 instead of 0. the root is stored at $X[1]$. For a node stored at $X[i]$, the left child, if any, is stored in $X[2i]$ and the right child, if any, in $X[2i+1]$. To be able to store any binary tree on n vertices the minimum size of X should be. (GATE CS 2006)

- (A) $\log_2 n$
- (B) n
- (C) $2n + 1$
- (D) $2^n - 1$

Answer (D)

For a right skewed binary tree, number of nodes will be $2^n - 1$. For example, in below binary tree, node 'A' will be stored at index 1, 'B' at index 3, 'C' at index 7 and 'D' at index 15.





3. Which one of the following in place sorting algorithms needs the minimum number of swaps? (GATE CS 2006)

- (A) Quick sort**
- (B) Insertion sort**
- (C) Selection sort**
- (D) Heap sort**

Answer (C)

For selection sort, number of swaps required is minimum ($O(n)$).

4. An element in an array X is called a leader if it is greater than all elements to the right of it in X. The best algorithm to find all leaders in an array (GATE CS 2006)

- (A) Solves it in linear time using a left to right pass of the array**
- (B) Solves it in linear time using a right to left pass of the array**
- (C) Solves it using divide and conquer in time $8(n \log n)$**
- (D) Solves it in time $8(n^2)$**

Answer (B)

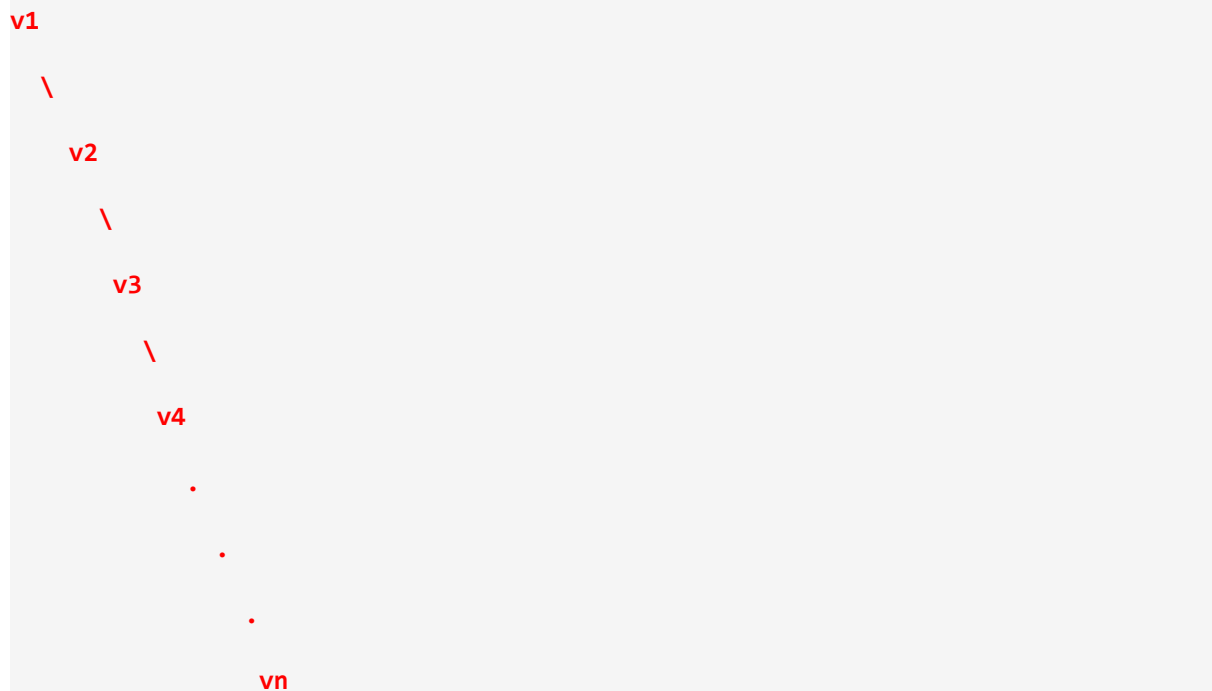
Please see [this post](#) for explanation.

5. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is: (GATE CS 2006)

- (A) $n - 1$**
- (B) $2n - 2$**
- (C) nC_2**
- (D) 2**

Answer (B)

Minimum spanning tree of such a graph is



Weight of the minimum spanning tree

$$= 2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| \dots + 2|n - (n-1)|$$
$$= 2n - 2$$

Following questions have been asked in GATE CS exam.

1. The usual $\Theta(n^2)$ implementation of Insertion Sort to sort an array uses linear search to identify the position where an element is to be inserted into the already sorted part of the array. If, instead, we use binary search to identify the position, the worst case running time will (GATE CS 2003)

- (a) remain $\Theta(n^2)$**
- (b) become $\Theta(n(\log n)^2)$**
- (c) become $\Theta(n \log n)$**
- (d) become $\Theta(n)$**

Answer (a)

If we use binary search then there will be $\lceil \log_2(n!) \rceil$ comparisons in the worst case, which is $\Theta(n \log n)$ (If you want to know how $\lceil \log_2(n!) \rceil$ can be equal to $\Theta(n \log n)$), then see [this](#) for proof). But the algorithm as a whole will still have a running time of $\Theta(n^2)$ on average because of the series of swaps required for each insertion.

Reference:

http://en.wikipedia.org/wiki/Insertion_sort

2. The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

- a) n
- b) n^2
- c) $n \log n$
- d) $n(\log^2 n)$

Answer (c)

The number of comparisons that a comparison sort algorithm requires increases in proportion to $n \log(n)$, where n is the number of elements to sort. This bound is asymptotically tight:

Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are n factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most $f(n)$ steps, it cannot distinguish more than $2^{f(n)}$ cases because the keys are distinct and each comparison has only two possible outcomes. Therefore,

$$2^{f(n)} \geq n!, \text{ or equivalently } f(n) \geq \log_2(n!).$$

References:

http://en.wikipedia.org/wiki/Comparison_sort

http://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15451-s07/www/lecture_notes/lect0130.pdf

3. The problem 3-SAT and 2-SAT are

- a) both in P
- b) both NP complete
- c) NP-complete and in P respectively
- d) undecidable and NP-complete respectively

Answer (c)

The Boolean satisfiability problem (SAT) is a decision problem, whose instance is a Boolean expression written using only AND, OR, NOT, variables, and parentheses. The problem is: given the expression, is there some assignment of TRUE and FALSE values to the variables that will make the entire expression true? A formula of propositional logic is said to be satisfiable if logical values can be assigned to its variables in a way that makes the formula true.

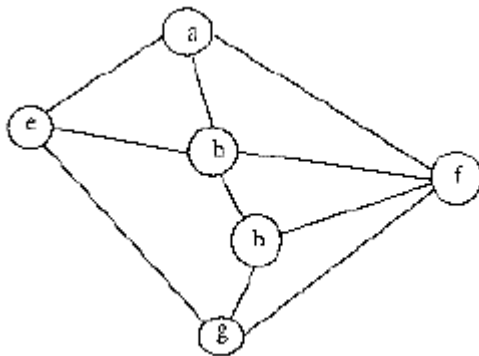
3-SAT and 2-SAT are special cases of k-satisfiability (k-SAT) or simply satisfiability (SAT), when each clause contains exactly $k = 3$ and $k = 2$ literals respectively.

2-SAT is P while 3-SAT is NP Complete. (See [this](#) for explanation)

References:

http://en.wikipedia.org/wiki/Boolean_satisfiability_problem

4. Consider the following graph



Among the following sequences

I) a b e g h f

II) a b f e h g

III) a b f h g e

IV) a f g h b e

Which are depth first traversals of the above graph? (GATE CS 2003)

a) I, II and IV only

b) I and IV only

c) II, III and IV only

d) I, III and IV only

Answer (d)

Following questions have been asked in GATE CS exam.

1. Consider the following C function.

```
float f(float x, int y)
```

```
{  
  
    float p, s; int i;  
  
    for (s=1, p=1, i=1; i < y; i ++)  
  
    {  
  
        p*= x/i;  
  
        s+=p;  
  
    }  
  
    return s;  
  
}
```

Run on IDE

For large values of y , the return value of the function f best approximates (GATE CS 2003)

- a) x^y
- b) e^x
- c) $\ln(1 + x)$
- d) x^x

Answer (b)

The function $f()$ is implementation of Taylor's Series to calculates e^x

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

More is the value of y more precise value of e^x will be returned by $f()$

References:

http://en.wikipedia.org/wiki/E_%28mathematical_constant%29

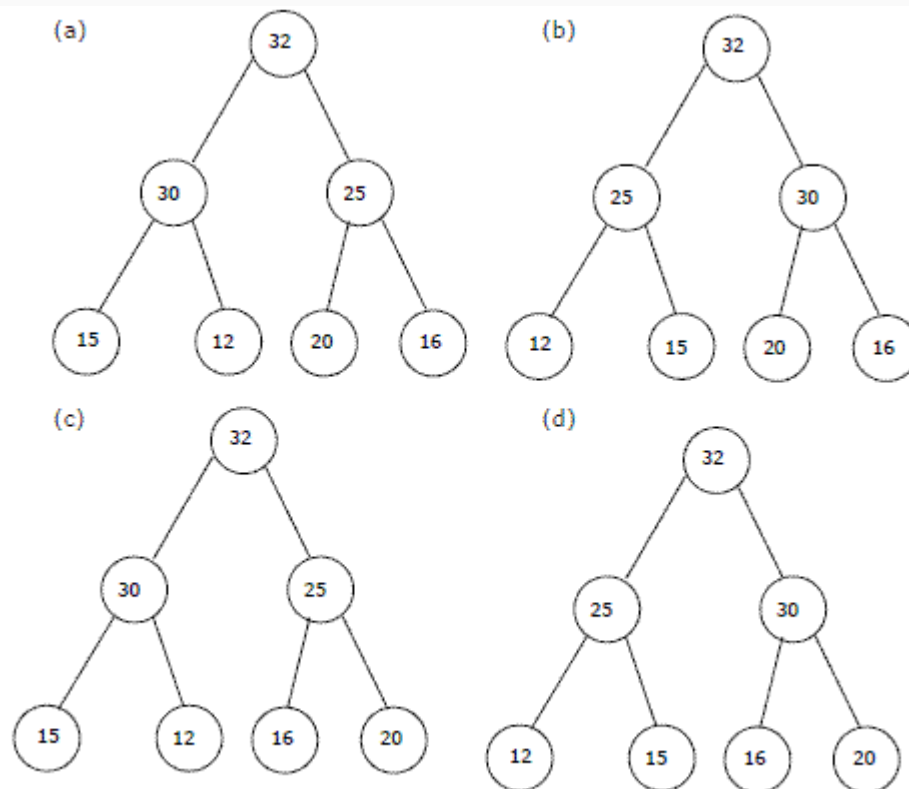
2. In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is (GATE CS 2002)

- a) $\log_2 n$
- b) $n/2$
- c) $\log_2 n - 1$
- d) n

Answer(d)

In the worst case, the element to be searched has to be compared with all elements of linked list.

3. The elements 32, 15, 20, 30, 12, 25, 16 are inserted one by one in the given order into a Max Heap. The resultant Max Heap is.



Answer (a)

4. Consider the following three claims

I $(n + k)^m = \Theta(n^m)$, where k and m are constants

II $2^{(n+1)} = O(2^n)$

III $2^{(2n+1)} = O(2^n)$

Which of these claims are correct? (GATE CS 2003)

- (a) I and II
- (b) I and III
- (c) II and III
- (d) I, II and III

Answer(a)

(I) $(n+m)^k = n^k + c_1 n^{(k-1)} + \dots + k^m = \theta(n^k)$

(II) $2^{(n+1)} = 2 \cdot 2^n = O(2^n)$

5. A single array A[1..MAXSIZE] is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables top1 and top2 (top1 < top2) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for “stack full” is (GATE CS 2004)

- a) (top1 = MAXSIZE/2) and (top2 = MAXSIZE/2+1)
- b) top1 + top2 = MAXSIZE
- c) (top1 = MAXSIZE/2) or (top2 = MAXSIZE)
- d) top1 = top2 - 1

Answer(d)

If we are to use space efficiently then size of the any stack can be more than MAXSIZE/2.

Both stacks will grow from both ends and if any of the stack top reaches near to the other top then stacks are full. So the condition will be top1 = top2 - 1 (given that top1 < top2)

1. Consider the following C program segment

```
struct CellNode
{
    struct CellNode *leftchild;
    int element;
```

```
struct CelINode *rightChild;
```

```
}
```

```
int Dosomething(struct CelINode *ptr)
```

```
{
```

```
    int value = 0;
```

```
    if (ptr != NULL)
```

```
    {
```

```
        if (ptr->leftChild != NULL)
```

```
            value = 1 + Dosomething(ptr->leftChild);
```

```
        if (ptr->rightChild != NULL)
```

```
            value = max(value, 1 + Dosomething(ptr->rightChild));
```

```
    }
```

```
    return (value);
```

}

Run on IDE

The value returned by the function `DoSomething` when a pointer to the root of a non-empty tree is passed as argument is (GATE CS 2004)

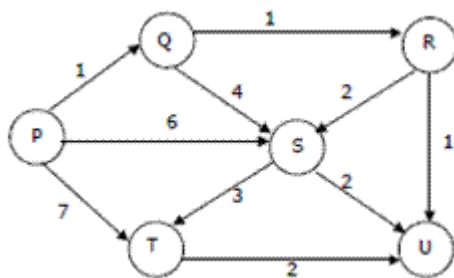
- a) The number of leaf nodes in the tree
- b) The number of nodes in the tree
- c) The number of internal nodes in the tree
- d) The height of the tree

Answer: (d)

Explanation: `DoSomething()` returns $\max(\text{height of left child} + 1, \text{height of right child} + 1)$. So given that pointer to root of tree is passed to `DoSomething()`, it will return height of the tree. Note that this implementation follows the convention where height of a single node is 0.

2. Suppose we run Dijkstra's single source shortest-path algorithm on the following edge weighted directed graph with vertex P as the source. In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized? (GATE CS 2004)

- a) P, Q, R, S, T, U
- b) P, Q, R, U, S, T
- c) P, Q, R, U, T, S
- d) P, Q, T, R, U, S



Answer (b)

3. Suppose each set is represented as a linked list with elements in arbitrary order. Which of the operations among union, intersection, membership, cardinality will be the slowest? (GATE CS 2004)

- a) union only
- b) intersection, membership

- c) membership, cardinality
- d) union, intersection

Answer (d)

Cardinality and membership are definitely not the slowest one. For cardinality, just count the number of nodes in a list. For membership, just traverse the list and look for a match

For getting intersection of L1 and L2, search for each element of L1 in L2 and print the elements we find in L2.

There can be many ways for getting union of L1 and L2. One of them is as follows

- a) Print all the nodes of L1 and print only those which are not present in L2.**
- b) Print nodes of L2.**

4. The time complexity of the following C function is (assume $n > 0$) (GATE CS 2004)

```
int recursive (mt n)
{
    if (n == 1)
        return (1);
    else
        return (recursive (n-1) + recursive (n-1));
}
```

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) $O(2^n)$

Answer(d)

Explanation:

Recursive expression for the above program will be.

$$T(n) = 2T(n-1) + c$$

$$T(1) = c1.$$

Let us solve it.

$$T(n) = 2(2T(n-2) + c) + c = 4T(n-2) + 3c$$

$$T(n) = 8T(n-3) + 6c + c = 8T(n-3) + 7c$$

$$T(n) = 16T(n-4) + 14c + c = 16T(n-4) + 15c$$

.....

.....

$$T(n) = (2^{(n-1)})T(1) + (2^{(n-1)} - 1)c$$

$$T(n) = O(2^n)$$

Following questions have asked in GATE CS exam.

1. Suppose you are given an array $s[1..n]$ and a procedure reverse (s,i,j) which reverses the order of elements in a between positions i and j (both inclusive). What does the following sequence

do, where $1 < k \leq n$:

reverse ($s, 1, k$);

reverse ($s, k + 1, n$);

reverse ($s, 1, n$);

(GATE CS 2000)

(a) Rotates s left by k positions

(b) Leaves s unchanged

(c) Reverses all elements of s

(d) None of the above

Answer: (a)

Effect of the above 3 reversals for any k is equivalent to left rotation of the array of size n by k . Please see [this post](#) for details.

If we rotate an array n times for $k = 1$ to n , we get the same array back.

2. The best data structure to check whether an arithmetic expression has balanced parentheses is a (GATE CS 2004)

- a) queue
- b) stack
- c) tree
- d) list

Answer(b)

There are three types of parentheses [] { } (). Below is an arbitrary code segment which has parentheses of all three types.

```
void func(int c, int a[])  
  
{  
  
    return ((c +2) + arr[(c-2)]) ;  
  
}
```

Run on IDE

Stack is a straightforward choice for checking if left and right parentheses are balanced. Here is a algorithm to do the same.

```
/*Return 1 if expression has balanced parentheses */  
  
bool areParenthesesBalanced(expression )  
  
{  
  
    for each character in expression
```

```
{
```

```
if(character == '(' || character == '{' || character == '[')
```

```
push(stack, character);
```

```
if(character == ')') || character == '}' || character == ']')
```

```
{
```

```
if(isEmpty(stack))
```

```
return 0; /*We are seeing a right parenthesis
```

```
without a left pair*/
```

```
/* Pop the top element from stack, if it is not a pair
```

```
bracket of character then there is a mismatch.
```

```
This will happen for expressions like {({}) */
```

```
else if (! isMatchingPair(pop(stack), character) )
```

```
return 0;
```

```
}  
  
}  
  
if(isEmpty(stack))  
  
    return 1; /*balanced*/  
  
else  
  
    return 0; /*not balanced*/  
  
} /* End of function to check parentheses */  
  
/* Returns 1 if character1 and character2 are matching left  
and right parentheses */  
  
bool isMatchingPair(character1, character2)  
  
{  
  
    if(character1 == '(' && character2 == ')')
```

```
return 1;
```

```
else If(character1 == '{' && character2 == '}')
```

```
return 1;
```

```
else If(character1 == '[' && character2 == ']')
```

```
return 1;
```

```
else
```

```
return 0;
```

```
}
```

Run on IDE

3. Level order traversal of a rooted tree can be done by starting from the root and performing (GATE CS 2004)

- a) preorder traversal
- b) in-order traversal
- c) depth first search
- d) breadth first search

Answer(d)

See [this post](#) for details

4. Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function $x \bmod 10$, which of the following statements are true?

- i. 9679, 1989, 4199 hash to the same value
- ii. 1471, 6171 has to the same value
- iii. All elements hash to the same value

**iv. Each element hashes to a different value
(GATE CS 2004)**

- a) i only**
- b) ii only**
- c) i and ii only**
- d) iii or iv**

Answer (c)

5. Postorder traversal of a given binary search tree, T produces the following sequence of keys

10, 9, 23, 22, 27, 25, 15, 50, 95, 60, 40, 29

Which one of the following sequences of keys can be the result of an in-order traversal of the tree T? (GATE CS 2005)

- a) 9, 10, 15, 22, 23, 25, 27, 29, 40, 50, 60, 95**
- b) 9, 10, 15, 22, 40, 50, 60, 95, 23, 25, 27, 29**
- c) 29, 15, 9, 10, 25, 22, 23, 27, 40, 60, 50, 95**
- d) 95, 50, 60, 40, 27, 23, 22, 25, 10, 9, 15, 29**

Answer (a)